

COMP132: Advanced Programming
Programming Project Report

Knowledge Siege: Simulation Design and Development

Ahmet Buğra Ertürk, 86877

<Spring 2025>



1. Pre-Registered Player

a. Username and Password

- i. Username: ahmetbugra, Password: 123456
- ii. Username: bedirhankokduman, Password: 000000
- iii. Username: gürayk, Password: gk20
- iv. Username: halilibo, Password: halil123

b. Log in with pre-registered users

You can log in with the login screen in Figure 1 using username and password.

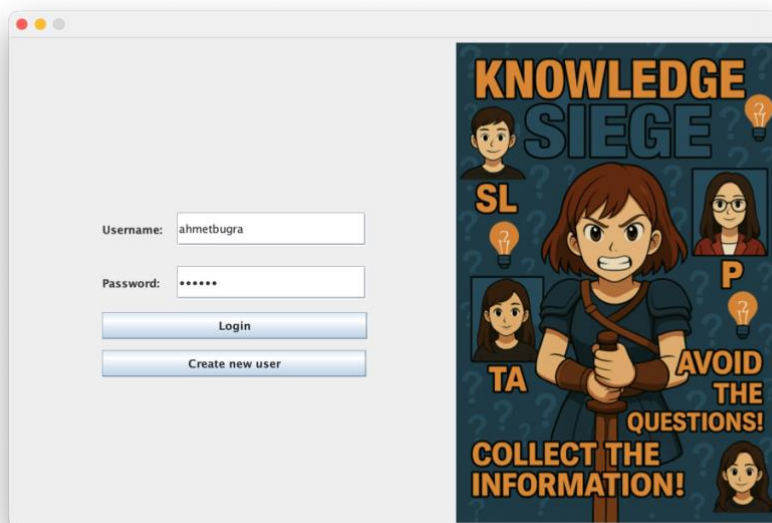


Figure 1 - Login Screen with “Username” and “Password” fields, and “Login” and “Create New User” button.

c. Errors

- i. Empty fields

In the case of clicking login button with empty fields, dialog in the Figure 2 is showed.

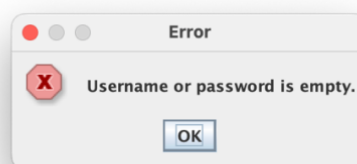


Figure 2 – Error Dialog for empty fields

ii. Invalid username or password

If the username does not exist or username and password does not match, dialog in the Figure 3 is showed.

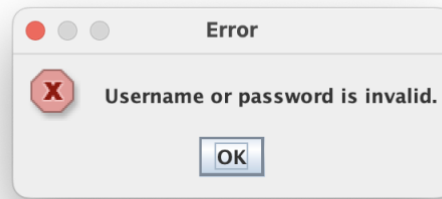


Figure 3 – Error Dialog for invalid username or password

d. Logs

You can see the logs of each pre-registered player as text. They are stored in `[username].log` file under the `users/[username]` folder.¹

i. ahmetbugra

Account has been created at 24.05.2025 16.35.05

User logged in at 24.05.2025 16.35.23

New game started at 24.05.2025 16.36.05

10 point collected, new score is 10 at 24.05.2025 16.36.11
10 point collected, new score is 20 at 24.05.2025 16.36.12
5 damage taken, new health is 95 at 24.05.2025 16.36.13
10 point collected, new score is 30 at 24.05.2025 16.36.14
10 point collected, new score is 40 at 24.05.2025 16.36.15
10 point collected, new score is 50 at 24.05.2025 16.36.17
Passed to level 2 at 24.05.2025 16.36.18

20 point collected, new score is 20 at 24.05.2025 16.36.22
10 point collected, new score is 30 at 24.05.2025 16.36.23
10 point collected, new score is 40 at 24.05.2025 16.36.25
20 point collected, new score is 60 at 24.05.2025 16.36.25
10 point collected, new score is 70 at 24.05.2025 16.36.26
10 point collected, new score is 80 at 24.05.2025 16.36.28
10 point collected, new score is 90 at 24.05.2025 16.36.29
20 point collected, new score is 110 at 24.05.2025 16.36.30
Passed to level 3 at 24.05.2025 16.36.31

20 point collected, new score is 20 at 24.05.2025 16.36.35
20 point collected, new score is 40 at 24.05.2025 16.36.36
20 point collected, new score is 60 at 24.05.2025 16.36.37
30 point collected, new score is 90 at 24.05.2025 16.36.39
20 point collected, new score is 110 at 24.05.2025 16.36.40
30 point collected, new score is 140 at 24.05.2025 16.36.41
20 point collected, new score is 160 at 24.05.2025 16.36.43
The game is succesfully done with total score 320 at 24.05.2025 16.36.43

¹ In this report, square brackets ([]) will be used to show variable names.

New game started at 24.05.2025 16.37.01

10 point collected, new score is 10 at 24.05.2025 16.37.06
10 point collected, new score is 20 at 24.05.2025 16.37.07
10 point collected, new score is 30 at 24.05.2025 16.37.09
10 point collected, new score is 40 at 24.05.2025 16.37.10
10 point collected, new score is 50 at 24.05.2025 16.37.11
Passed to level 2 at 24.05.2025 16.37.12

5 damage taken, new health is 95 at 24.05.2025 16.37.17
10 damage taken, new health is 85 at 24.05.2025 16.37.17
10 damage taken, new health is 75 at 24.05.2025 16.37.18
5 damage taken, new health is 70 at 24.05.2025 16.37.18
5 damage taken, new health is 65 at 24.05.2025 16.37.19
5 damage taken, new health is 60 at 24.05.2025 16.37.21
10 damage taken, new health is 50 at 24.05.2025 16.37.21
5 damage taken, new health is 45 at 24.05.2025 16.37.22
5 damage taken, new health is 40 at 24.05.2025 16.37.23
5 damage taken, new health is 35 at 24.05.2025 16.37.25
5 damage taken, new health is 30 at 24.05.2025 16.37.26
10 damage taken, new health is 20 at 24.05.2025 16.37.29
10 damage taken, new health is 10 at 24.05.2025 16.37.29
5 damage taken, new health is 5 at 24.05.2025 16.37.29
5 damage taken, new health is 0 at 24.05.2025 16.37.30
The game is over with the total score 50 at 24.05.2025 16.37.30

ii. bedirhankokduman

Account has been created at 24.05.2025 13.00.03

User logged in at 24.05.2025 13.00.12

New game started at 24.05.2025 13.00.15

10 point collected, new score is 10 at 24.05.2025 13.00.20
10 point collected, new score is 20 at 24.05.2025 13.00.21
10 point collected, new score is 30 at 24.05.2025 13.00.21
10 point collected, new score is 40 at 24.05.2025 13.00.22
10 point collected, new score is 50 at 24.05.2025 13.00.24
Passed to level 2 at 24.05.2025 13.00.25

10 point collected, new score is 10 at 24.05.2025 13.00.30
20 point collected, new score is 30 at 24.05.2025 13.00.31
20 point collected, new score is 50 at 24.05.2025 13.00.31
10 point collected, new score is 60 at 24.05.2025 13.00.32
5 damage taken, new health is 95 at 24.05.2025 13.00.32
10 damage taken, new health is 85 at 24.05.2025 13.00.35
5 damage taken, new health is 80 at 24.05.2025 13.00.35
10 point collected, new score is 70 at 24.05.2025 13.00.35
10 damage taken, new health is 70 at 24.05.2025 13.00.37
10 damage taken, new health is 60 at 24.05.2025 13.00.38
5 damage taken, new health is 55 at 24.05.2025 13.00.39
20 point collected, new score is 90 at 24.05.2025 13.00.40
5 damage taken, new health is 50 at 24.05.2025 13.00.41
5 damage taken, new health is 45 at 24.05.2025 13.00.41
20 point collected, new score is 110 at 24.05.2025 13.00.43
Passed to level 3 at 24.05.2025 13.00.44

20 damage taken, new health is 80 at 24.05.2025 13.00.47
20 damage taken, new health is 60 at 24.05.2025 13.00.48
10 damage taken, new health is 50 at 24.05.2025 13.00.49
20 point collected, new score is 20 at 24.05.2025 13.00.49

20 damage taken, new health is 30 at 24.05.2025 13.00.50
20 damage taken, new health is 10 at 24.05.2025 13.00.51
20 point collected, new score is 40 at 24.05.2025 13.00.51
20 damage taken, new health is -10 at 24.05.2025 13.00.53
The game is over with the total score 200 at 24.05.2025 13.00.53

User logged in at 24.05.2025 13.09.32

New game started at 24.05.2025 13.09.33

10 point collected, new score is 10 at 24.05.2025 13.09.39
10 point collected, new score is 20 at 24.05.2025 13.09.40
10 point collected, new score is 30 at 24.05.2025 13.09.41
10 point collected, new score is 40 at 24.05.2025 13.09.44
10 point collected, new score is 50 at 24.05.2025 13.09.46
Passed to level 2 at 24.05.2025 13.09.48

20 point collected, new score is 20 at 24.05.2025 13.09.51
10 point collected, new score is 30 at 24.05.2025 13.09.54
10 point collected, new score is 40 at 24.05.2025 13.09.54
20 point collected, new score is 60 at 24.05.2025 13.09.57
10 damage taken, new health is 90 at 24.05.2025 13.09.57
5 damage taken, new health is 85 at 24.05.2025 13.09.58
20 point collected, new score is 80 at 24.05.2025 13.10.01
10 point collected, new score is 90 at 24.05.2025 13.10.02
10 point collected, new score is 100 at 24.05.2025 13.10.03
Passed to level 3 at 24.05.2025 13.10.04

10 damage taken, new health is 90 at 24.05.2025 13.10.07
30 point collected, new score is 30 at 24.05.2025 13.10.08
20 point collected, new score is 50 at 24.05.2025 13.10.08
10 damage taken, new health is 80 at 24.05.2025 13.10.10
30 point collected, new score is 80 at 24.05.2025 13.10.10
20 damage taken, new health is 60 at 24.05.2025 13.10.11
10 damage taken, new health is 50 at 24.05.2025 13.10.11
20 damage taken, new health is 30 at 24.05.2025 13.10.13
10 damage taken, new health is 20 at 24.05.2025 13.10.13
20 damage taken, new health is 0 at 24.05.2025 13.10.14
The game is over with the total score 230 at 24.05.2025 13.10.14

iii. gürayk

Account has been created at 24.05.2025 13.01.38

User logged in at 24.05.2025 13.01.45

New game started at 24.05.2025 13.01.48

10 point collected, new score is 10 at 24.05.2025 13.01.53
10 point collected, new score is 20 at 24.05.2025 13.01.54
10 point collected, new score is 30 at 24.05.2025 13.01.54
10 point collected, new score is 40 at 24.05.2025 13.01.56
5 damage taken, new health is 95 at 24.05.2025 13.01.57
10 point collected, new score is 50 at 24.05.2025 13.01.58
Passed to level 2 at 24.05.2025 13.01.59

20 point collected, new score is 20 at 24.05.2025 13.02.04
10 point collected, new score is 30 at 24.05.2025 13.02.05
10 point collected, new score is 40 at 24.05.2025 13.02.05
10 point collected, new score is 50 at 24.05.2025 13.02.06
10 damage taken, new health is 90 at 24.05.2025 13.02.06
10 point collected, new score is 60 at 24.05.2025 13.02.08
10 point collected, new score is 70 at 24.05.2025 13.02.09

10 point collected, new score is 80 at 24.05.2025 13.02.10
20 point collected, new score is 100 at 24.05.2025 13.02.11
Passed to level 3 at 24.05.2025 13.02.12

20 point collected, new score is 20 at 24.05.2025 13.02.15
20 point collected, new score is 40 at 24.05.2025 13.02.16
20 point collected, new score is 60 at 24.05.2025 13.02.17
20 point collected, new score is 80 at 24.05.2025 13.02.18
30 point collected, new score is 110 at 24.05.2025 13.02.19
20 point collected, new score is 130 at 24.05.2025 13.02.20
20 point collected, new score is 150 at 24.05.2025 13.02.21
The game is succesfully done with total score 300 at 24.05.2025
13.02.21

User logged in at 24.05.2025 13.08.33

New game started at 24.05.2025 13.08.35

10 point collected, new score is 10 at 24.05.2025 13.08.41
10 point collected, new score is 20 at 24.05.2025 13.08.41
5 damage taken, new health is 95 at 24.05.2025 13.08.42
10 point collected, new score is 30 at 24.05.2025 13.08.42
10 point collected, new score is 40 at 24.05.2025 13.08.44
10 point collected, new score is 50 at 24.05.2025 13.08.45
Passed to level 2 at 24.05.2025 13.08.46

10 damage taken, new health is 90 at 24.05.2025 13.08.51
5 damage taken, new health is 85 at 24.05.2025 13.08.51
5 damage taken, new health is 80 at 24.05.2025 13.08.52
20 point collected, new score is 20 at 24.05.2025 13.08.52
10 point collected, new score is 30 at 24.05.2025 13.08.52
10 damage taken, new health is 70 at 24.05.2025 13.08.55
10 damage taken, new health is 60 at 24.05.2025 13.08.55
20 point collected, new score is 50 at 24.05.2025 13.08.57
5 damage taken, new health is 55 at 24.05.2025 13.08.57
10 damage taken, new health is 45 at 24.05.2025 13.08.58
5 damage taken, new health is 40 at 24.05.2025 13.08.59
10 damage taken, new health is 30 at 24.05.2025 13.09.00
10 point collected, new score is 60 at 24.05.2025 13.09.00
10 damage taken, new health is 20 at 24.05.2025 13.09.02
10 damage taken, new health is 10 at 24.05.2025 13.09.02
10 point collected, new score is 70 at 24.05.2025 13.09.03
10 damage taken, new health is 0 at 24.05.2025 13.09.05
The game is over with the total score 120 at 24.05.2025 13.09.05

iv. halilibo

Account has been created at 24.05.2025 13.03.02

User logged in at 24.05.2025 13.03.14

New game started at 24.05.2025 13.03.16

10 point collected, new score is 10 at 24.05.2025 13.03.21
10 point collected, new score is 20 at 24.05.2025 13.03.23
5 damage taken, new health is 95 at 24.05.2025 13.03.25
10 point collected, new score is 30 at 24.05.2025 13.03.27
10 point collected, new score is 40 at 24.05.2025 13.03.28
10 point collected, new score is 50 at 24.05.2025 13.03.30
Passed to level 2 at 24.05.2025 13.03.31

20 point collected, new score is 20 at 24.05.2025 13.03.48
10 point collected, new score is 30 at 24.05.2025 13.03.52

20 point collected, new score is 50 at 24.05.2025 13.03.53
10 damage taken, new health is 90 at 24.05.2025 13.03.54
10 point collected, new score is 60 at 24.05.2025 13.03.54
10 point collected, new score is 70 at 24.05.2025 13.04.01
20 point collected, new score is 90 at 24.05.2025 13.04.01
20 point collected, new score is 110 at 24.05.2025 13.04.04
Passed to level 3 at 24.05.2025 13.04.05

10 damage taken, new health is 90 at 24.05.2025 13.04.18
20 point collected, new score is 20 at 24.05.2025 13.04.19
20 point collected, new score is 40 at 24.05.2025 13.04.20
30 point collected, new score is 70 at 24.05.2025 13.04.22
30 point collected, new score is 100 at 24.05.2025 13.04.26
20 point collected, new score is 120 at 24.05.2025 13.04.38
30 point collected, new score is 150 at 24.05.2025 13.04.38
The game is succesfully done with total score 310 at 24.05.2025
13.04.38

Unsuccessful login attempt at 24.05.2025 13.10.35

User logged in at 24.05.2025 13.10.39

New game started at 24.05.2025 13.10.41

10 point collected, new score is 10 at 24.05.2025 13.10.46
10 point collected, new score is 20 at 24.05.2025 13.10.46
10 point collected, new score is 30 at 24.05.2025 13.10.50
10 point collected, new score is 40 at 24.05.2025 13.10.50
10 point collected, new score is 50 at 24.05.2025 13.10.52
Passed to level 2 at 24.05.2025 13.10.53

20 point collected, new score is 20 at 24.05.2025 13.10.57
10 point collected, new score is 30 at 24.05.2025 13.10.58
10 point collected, new score is 40 at 24.05.2025 13.10.59
10 point collected, new score is 50 at 24.05.2025 13.11.00
5 damage taken, new health is 95 at 24.05.2025 13.11.00
5 damage taken, new health is 90 at 24.05.2025 13.11.02
5 damage taken, new health is 85 at 24.05.2025 13.11.03
20 point collected, new score is 70 at 24.05.2025 13.11.04
5 damage taken, new health is 80 at 24.05.2025 13.11.05
5 damage taken, new health is 75 at 24.05.2025 13.11.06
5 damage taken, new health is 70 at 24.05.2025 13.11.07
5 damage taken, new health is 65 at 24.05.2025 13.11.07
5 damage taken, new health is 60 at 24.05.2025 13.11.10
10 damage taken, new health is 50 at 24.05.2025 13.11.12
10 point collected, new score is 80 at 24.05.2025 13.11.13
20 point collected, new score is 100 at 24.05.2025 13.11.13
Passed to level 3 at 24.05.2025 13.11.14

20 damage taken, new health is 80 at 24.05.2025 13.11.18
20 damage taken, new health is 60 at 24.05.2025 13.11.18
10 damage taken, new health is 50 at 24.05.2025 13.11.19
20 point collected, new score is 20 at 24.05.2025 13.11.19
20 damage taken, new health is 30 at 24.05.2025 13.11.20
30 point collected, new score is 50 at 24.05.2025 13.11.21
10 damage taken, new health is 20 at 24.05.2025 13.11.21
20 damage taken, new health is 0 at 24.05.2025 13.11.23
The game is over with the total score 200 at 24.05.2025 13.11.23

2. New Player

In this section, creating new player and playing game are explained with using the creating process of the one of the pre-registered players, who is **ahmetbugra**.

a. Register

Register screen in Figure 4 can be opened by **Create New User** button in Login screen (Figure 1). To register, player need to enter username and password in the appropriate fields and click on the **Image Choose** button to open JFileChooser [1] (in Figure 5) and select an image with jpg, jpeg or png extension and 5:6 ratio for optimal display. In the scenario of choosing no photo, registering continues with a default image.

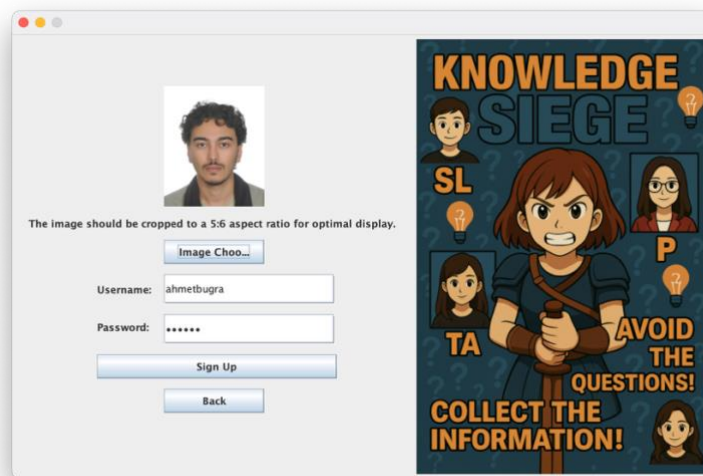


Figure 4 – Register Screen with preview of the chosen photo and required fields and buttons.

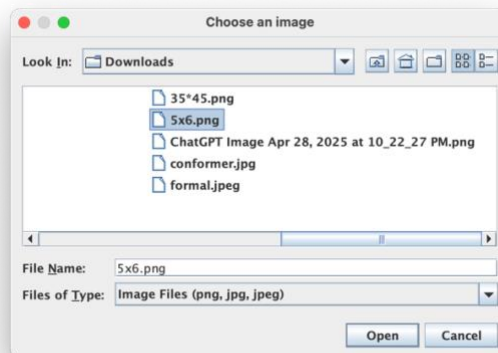


Figure 5 – JFileChooser to choose player image

b. Errors

- i. Trying to register with an existing username

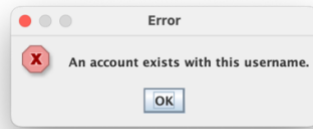


Figure 6 – Error Dialog for registering with an existing username

ii. Empty fields

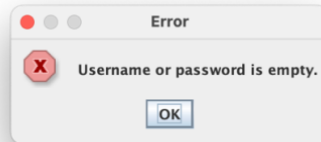


Figure 7 – Error Dialog for empty fields

iii. Containing comma (,)

This error occurs because the application uses a text file (`users.txt`) to store users' credentials and in this file, username and password are separated by comma.

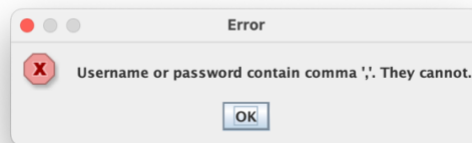


Figure 8 – Error Dialog for username and password containing comma

iv. Other cases

Some other cases such as file progresses can cause errors as well and they also show Error Dialogs, but I cannot show their Error Dialogs because I have not seen such a case yet. However, the application has the required codes to handle them.

c. Playing

i. The Game Rules

- The game has 3 level
- In level 1, randomly selected 4 SLs exist
- In level 2, remaining SLs and randomly selected 2 TAs exist
- In level 3, remaining TAs and all professors exist
- 50 points must be collected to pass level 2 from level 1
- 100 points must be collected to pass level 3 from level 2

- 150 points must be collected for victory
- If the health reach 0 in any level the game is over
- Score and health are reset through each level transition
- SLs' information boxes give 10 points
- TAs' information boxes give 20 points
- Professors' information boxes give 30 points
- SLs' question boxes give 5 damage
- TAs' question boxes give 10 damage
- Professors' question boxes give 20 damage
- SLs' movement direction is completely random
- TAs' movement direction is 40% towards player and 60% random
- Professors' movement direction is 80% towards player 20 % random
- Shooting info probability of SLs is 60% (40% for question)
- Shooting info probability of TAs is 50% (50% for question)
- Shooting info probability of professors is 25% (75% for question)
- The speeds of shotboxes are proportional to 3 4 and 5 for SLs TAs and professors respectively
- The final score is equal to sum of the collected point in each level
- Score and health is reset through each level transition.

ii. The Game Screens

After the registration, you can log in as mentioned before in **1.b**. Then, the home screen (in Figure 9) opens, and the **Play Game** button starts game from level 1

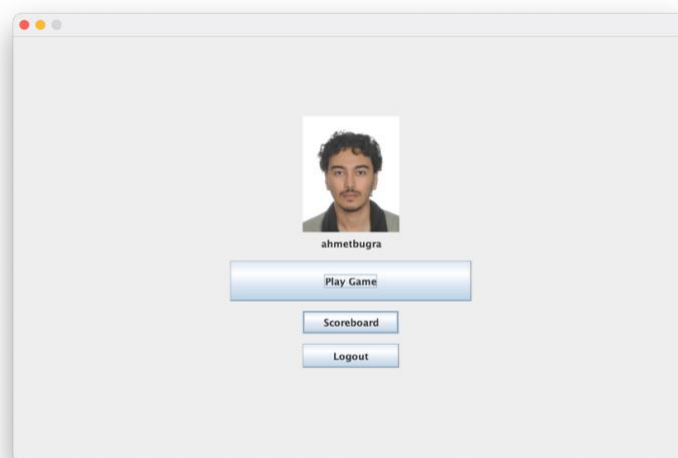


Figure 9 – Home Screen with profile picture, username and required buttons

In the Figure 10, the game has been started from level 1, and randomly selected 4 SLs, who also move randomly, and 1 player, who moves according to arrows on the keyboard, can be seen. Also, shotboxes are on the screen. Additionally, at the right side of the game, a health bar and a scoreboard exist, and there are two `JLabels` to show the information and questions which are hit by the player. And finally, there is a **Back** button to leave the game.

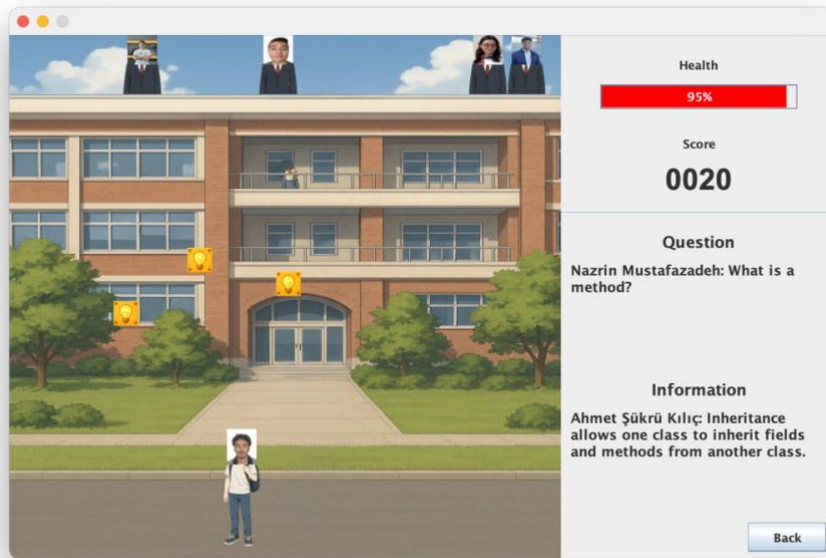


Figure 10 – The Game Screen for level 1

After collecting required points, the game passes to next level with the message dialogs in Figure 11 for different transitions, and the new level's game screens in Figure 12 opens with remaining SLs and randomly selected 2 TAs for level 2, and remaining TAs and all professors for level 3. Player's health and score are reset through each level transition.



Figure 11 – Message Dialog to show level transitions

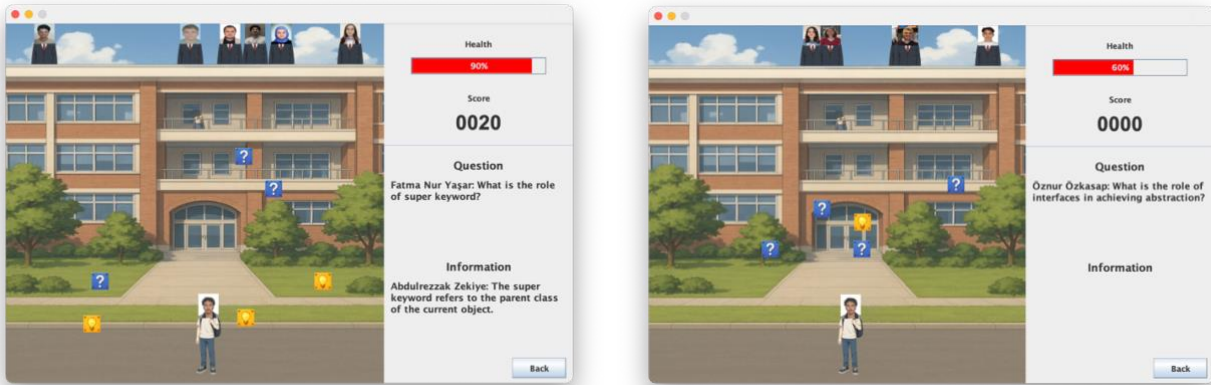


Figure 12 – The Game Screen for level 2 and level 3 respectively

The game is done when the 150 points are collected in level 3 and the total score levels is stored for scoreboard. If the health reach 0 in any level the game is over, and no score is saved. When finishing game, corresponding message dialog (in Figure 13) will be seen on the screen and after clicking **OK** the screen switching with the home screen.



Figure 13 – Message Dialogs to show game status

Also, while the player is moving right or left, the character image also turns this way as it can be seen in Figure 14.

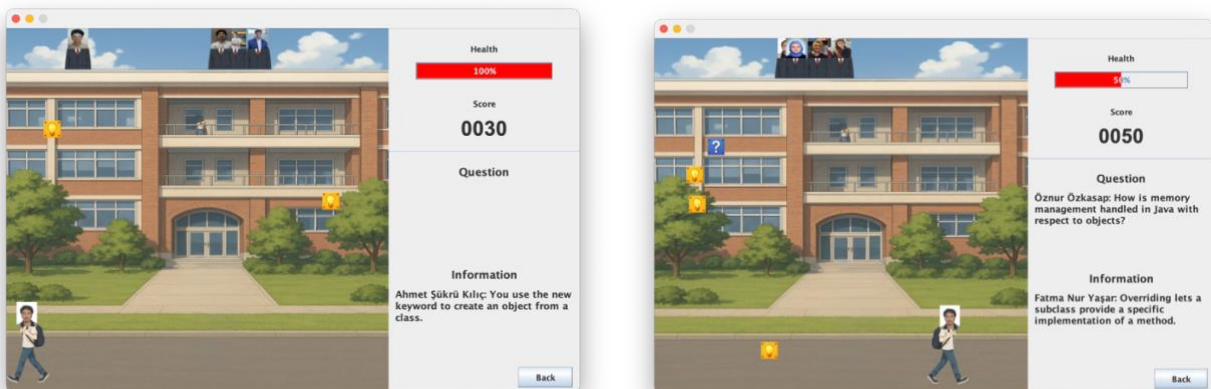
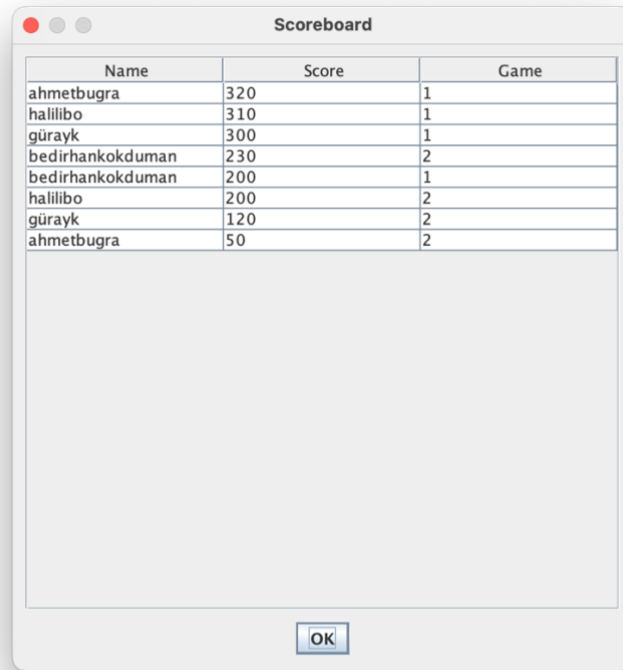


Figure 14 – The Game Screen with different directions of the player

Additionally, in the game screen, you can leave the game with **Back** button, and the line The player left the game at dd.mm.yyyy hh.mm.ss is written in the log file.

After finishing game, you can check the Scoreboard (Figure 15) with clicking the Scoreboard button in home screen (Figure 9).



Name	Score	Game
ahmetbugra	320	1
halilibo	310	1
gürayk	300	1
bedirhankokduman	230	2
bedirhankokduman	200	1
halilibo	200	2
gürayk	120	2
ahmetbugra	50	2

Figure 15 – The Scoreboard

iii. Log

The logs provided above contain 2 games, one of them is won, the other is over at level 2. This is the same as the log in section 1.d.i, because in this section, the creating process of the pre-registered player **ahmetbugra** is showed. After applying the required process for this section, the player remains a pre-registered player.

Account has been created at 24.05.2025 16.35.05

User logged in at 24.05.2025 16.35.23

New game started at 24.05.2025 16.36.05

10 point collected, new score is 10 at 24.05.2025 16.36.11
10 point collected, new score is 20 at 24.05.2025 16.36.12
5 damage taken, new health is 95 at 24.05.2025 16.36.13
10 point collected, new score is 30 at 24.05.2025 16.36.14
10 point collected, new score is 40 at 24.05.2025 16.36.15
10 point collected, new score is 50 at 24.05.2025 16.36.17
Passed to level 2 at 24.05.2025 16.36.18

20 point collected, new score is 20 at 24.05.2025 16.36.22

10 point collected, new score is 30 at 24.05.2025 16.36.23
10 point collected, new score is 40 at 24.05.2025 16.36.25
20 point collected, new score is 60 at 24.05.2025 16.36.25
10 point collected, new score is 70 at 24.05.2025 16.36.26
10 point collected, new score is 80 at 24.05.2025 16.36.28
10 point collected, new score is 90 at 24.05.2025 16.36.29
20 point collected, new score is 110 at 24.05.2025 16.36.30
Passed to level 3 at 24.05.2025 16.36.31

20 point collected, new score is 20 at 24.05.2025 16.36.35
20 point collected, new score is 40 at 24.05.2025 16.36.36
20 point collected, new score is 60 at 24.05.2025 16.36.37
30 point collected, new score is 90 at 24.05.2025 16.36.39
20 point collected, new score is 110 at 24.05.2025 16.36.40
30 point collected, new score is 140 at 24.05.2025 16.36.41
20 point collected, new score is 160 at 24.05.2025 16.36.43
The game is succesfully done with total score 320 at 24.05.2025
16.36.43

New game started at 24.05.2025 16.37.01

10 point collected, new score is 10 at 24.05.2025 16.37.06
10 point collected, new score is 20 at 24.05.2025 16.37.07
10 point collected, new score is 30 at 24.05.2025 16.37.09
10 point collected, new score is 40 at 24.05.2025 16.37.10
10 point collected, new score is 50 at 24.05.2025 16.37.11
Passed to level 2 at 24.05.2025 16.37.12

5 damage taken, new health is 95 at 24.05.2025 16.37.17
10 damage taken, new health is 85 at 24.05.2025 16.37.17
10 damage taken, new health is 75 at 24.05.2025 16.37.18
5 damage taken, new health is 70 at 24.05.2025 16.37.18
5 damage taken, new health is 65 at 24.05.2025 16.37.19
5 damage taken, new health is 60 at 24.05.2025 16.37.21
10 damage taken, new health is 50 at 24.05.2025 16.37.21
5 damage taken, new health is 45 at 24.05.2025 16.37.22
5 damage taken, new health is 40 at 24.05.2025 16.37.23
5 damage taken, new health is 35 at 24.05.2025 16.37.25
5 damage taken, new health is 30 at 24.05.2025 16.37.26
10 damage taken, new health is 20 at 24.05.2025 16.37.29
10 damage taken, new health is 10 at 24.05.2025 16.37.29
5 damage taken, new health is 5 at 24.05.2025 16.37.29
5 damage taken, new health is 0 at 24.05.2025 16.37.30
The game is over with the total score 50 at 24.05.2025 16.37.30

Part 2

A. Packages, Classes, and Interfaces²

My packages, classes, and interfaces are given as a diagram in Figure 16. You can see the legend of this diagram from the bottom right.

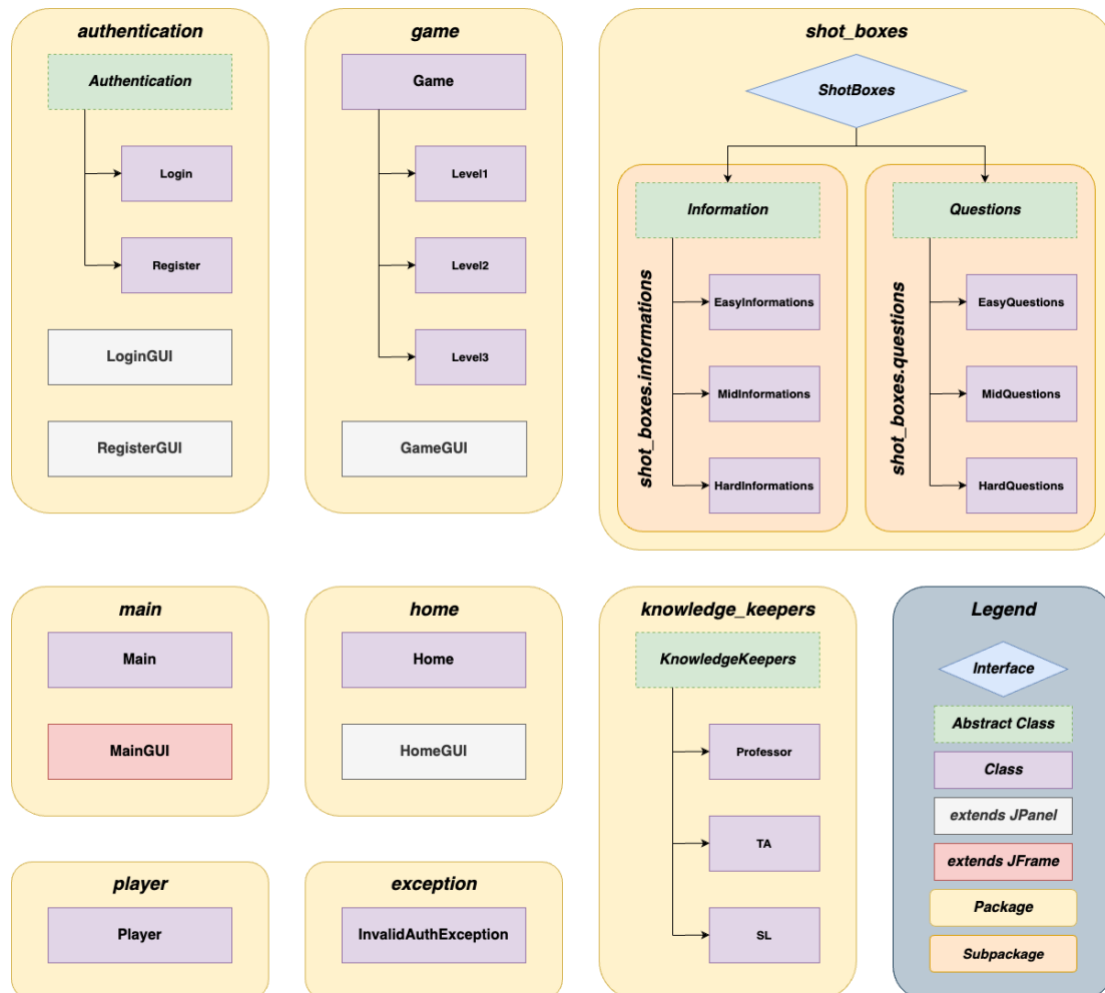


Figure 16 – Diagram of Packages, Classes, and Interfaces

1. main package

a. Main Class

In this class, there are three predefined `static final String[]` arrays to store names of knowledge keepers separately as SLs, TAs, and professors. Additionally it has another predefined `String[]` array to store possible extensions of an image (png, jpg, and jpeg)

² In this section, I will mention only the methods and classes which are written by myself, which means I will not explain classes in the default Java package and classical getter and setter methods.

Also, this class contains main method of the application. In this method, questions and information are uploaded by a static method with a `String` parameter to path of `.txt` file of questions and information in the `Questions` and `Information` abstract classes, so `Main` class has a **uses-a relationship** with `Questions` class and `Information` class.

It also creates `SL`, `TA`, and `Professor` objects with using predefined arrays. To do it, we iterate these arrays separately, then we also iterate extensions' array in this iteration, since we use images of knowledge keepers, but we do not know their image extensions, but we know that their images are stored with the name of their own name. Therefore, in these iterations we can check whether the file exists at the path `"src/assets/keepers/" + name + "." + ext`. If it exists a `SL`, `TA`, or `Professor` object is created corresponding to iterated array. Finally, in this method, a static method `MainGUI.call()` is called to open the `JFrame` of the application.

`Main` class has a **uses-a relationship** with `MainGUI`, `SL`, `TA`, and `Professor` classes.

I also added the lines above at the beginning of main method to use default java GUI component instead of mac specified components and I take these lines from ChatGPT, since some components did not response the design parameter or methods of the components.

```
try {
    UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
} catch (Exception e) {
    e.printStackTrace();
}
```

b. MainGUI Class

This class extends `JFrame` class. The constructor of the class sets the size of the window and sets a `JPanel` object (`mainPanel`) whose layout is a `CardLayout` to the content pane of the `JFrame` with inherited `setContentPane()` [2] method. Also, other `JPanel` extended classes' objects (`LoginGUI`, `RegisterGUI`, `HomeGUI`, and 3 times `GameGUI` for each level) are created in the constructor and these objects are added to `mainPanel` with a corresponding `String` key (`"login"`, `"register"`, `"home"`, `"level1"`, `"level2"`, and `"level3"`) to use when switching screens. Finally, `LoginGUI` is showed with using this method `cardLayout.show(mainPanel, "login")` [2]. Therefore, `MainGUI` has **has-a relationship** with `LoginGUI`, `RegisterGUI`, `HomeGUI`, and `GameGUI`, also **is-a relationship** with `JFrame`.

This class also contains two other static method. One of them is `call()` method which is mentioned before at **section 1.a** creates `MainGUI` objects, so it sets visible the window. The other is `switchScreen(String key)`, which helps to switch between screens with the method `cardLayout.show(mainPanel, key)`.

2. authentication package

In this and following packages, I used `BufferedReader` and `BufferedWriter` to read and write files, since I have written this project before learning File I/O in the class, so I have learned it from another source [4].

a. Authentication (abstract) Class

This abstract class contains two static methods. One of them, which is `listExistUsers()`, returns the existing users which is read from `src/users/users.txt` as a `Map`, which is a **Java Collections Framework** type. The other one, which is `showErrorDialog(String msg)`, is only a shortcut to run `JOptionPane.showMessageDialog(null, msg, "Error", JOptionPane.ERROR_MESSAGE)` [3] when an exception occurs in the process of authentication.

b. Login Class

This class extends `Authentication` abstract class and contains only one static method `login(String username, String password)` which checks whether the value of the **inherited** `listExistUsers().get(username)` is the same as given password. If it is, a static method in `Home` class `Home.setPlayer(username)` is called, and the screen switches to the `HomeGUI` with `MainGUI.switchScreen("home")`. If it is not or one of the parameters is blank, the `InvalidAuthException`, which is a **custom exception**, is thrown with corresponding message, then a `try-catch` block handles it and `showErrorDialog(e.getMessage())` (`e` is the caught `InvalidAuthException`) is called. It also logs unsuccessful login attempts and successful logins.

Because of the inheritance, `Login` has **is-a relationship** with `Authentication` and **uses-a relationship** with `Home`, `MainGUI` and `InvalidAuthException` classes.

c. Register Class

This class extends `Authentication` abstract class and contains only one static method `register(String username, String password, File image)`. This method first checks some prerequisites of the registration such as whether the username

exists beforehand with the **inherited** method

`listExistUsers().containsKey(username)`, the username or password contains comma or empty. It should not contain comma since our `src/users/users.txt` file in which the users' credentials are stored separate the username and password with comma. If any prerequisites are not satisfied, an `InvalidAuthException` occurs, and a `try-catch` block handles it. Then, if all prerequisites are satisfied, first creates a new folder named as `[username]` under the `src/users/` folder, then the image file is copied to this folder with naming `[username].[png|jpg|jpeg]` and the `gameNum.txt` is created in `src/users/[username]/` folder and 0 is written in this file. If any error occurs in this progress, a `try-catch` block handles it, and in the `catch` part, all new created files are deleted, and **inherited** `showErrorDialog("Error while creating required folders and files. Try again.")` method is called. If there is no error, in finally block, the username and password are saved in the `src/users/users.txt`, and the method `Home.getPlayer().log("Account has been created", true)` is called to create and write a log file. At the end, the screen switches to login screen with the method `MainGUI.switchScreen("login")`.

Register class has **is-a relationship** with Authentication class and **uses-a relationship** with `InvalidAuthException`, `Home`, `Player`, and `MainGUI` classes.

d. LoginGUI Class

This class extends `JPanel` and uses `JLabel`, `JTextField`, `JPasswordField` and `JButton` GUI objects. In this panel, `JLabel` objects are used to show an image at the right of the screen and indicate the corresponding fields for username and password, and `JTextField` and `JPasswordField` objects are used to get username and password inputs. Also, there is 2 `JButton`; one of them **Login** button and it calls `Login.login(usernameField.getText(), new String(passwordField.getPassword()))` to login, and the other one is **Create New User** button, and it switches the screen to register screen with the method `MainGUI.switchScreen("register")`.

LoginGUI class has **is-a relationship** with `JPanel`, and **uses-a relationship** with `Login` and `MainGUI` classes

e. RegisterGUI Class

This class extends `JPanel` and uses 2 more Java GUI component in addition to `LoginGUI`. One of the additional components is a `JLabel` to show the preview of the chosen image and the other one is a `JButton` to open `JFileChooser [1]` to choose

image. In the case of choosing no image, the default image at the path `src/users/default.png` is used. Clicking **Sign Up** button calls the method `Register.register(usernameField.getText(), new String(passwordField.getPassword()), selectedFile)`, and the **Back** button calls the method `MainGUI.switchScreen("login")`.

`RegisterGUI` class has **is-a relationship** with `JPanel` and **uses-a relationship** with `Register` and `MainGUI`.

3. home package

a. Home Class

This class has `Player player`, `GameGUI currentGameGUI`, and `List<String[]> scores` objects to store respectively current player, current level's `GameGUI`, and final scores. To manage player object there are three method such that `setPlayer(String username)` which creates a `Player` object and assigns to `player`, `getPlayer()` which returns player object, and `resetPlayer()` which assigns null to `player`.

Also, this class manage the level transitions with `level1()`, `level2()`, and `level3()` methods which create corresponding `Game` extended objects such as `Level1`, `Level2`, and `Level3`, and assign corresponding public `GameGUI` object in `MainGUI` to `currentGameGUI`, finally switch the screen to `currentGameGUI`.

Additionally, this class is responsible for manage scoreboard. To do it, it has `readScores()` and `showScoreboard()` methods. The `readScores()` method first reads the `src/users/scores.txt` file and add the scores in `scores` List with the format `{[username], [score], [gameNum]}`, then sorts this list with ascending order for scores, then alphabetical order for username if the scores are same, and finally descending order for `gameNum` if the others are same. The other method `showScoreboard()` shows the scores in a `MessageDialog` as a `JTable` in a `JScrollPane` [5].

The `Home` class has **has-a relationship** with `Player` and `GameGUI` classes, and **uses-a relationship** with `Level1`, `Level2`, `Level3`, and `MainGUI` classes.

b. HomeGUI Class

This class extends `JPanel`, and has 2 `JLabel` to show player image from `Home.getPlayer().getPlayerIMG()` and username from `Home.getPlayer().getUsername()`, and 3 `JButton` to start game, logout, and open scoreboard. **Start Game** button calls `Home.getPlayer().resetScoreHealth()`, and

`Home.level1()` methods. **Logout** button calls `Home.resetPlayer()`, and `MainGUI.switchScreen("login")` methods. The **Scoreboard** button calls `Home.showScoreboard()` method.

Therefore, `HomeGUI` class has **is-a relationship** with `JPanel`, and **uses-a relationship** with `Home`, `Player`, and `MainGUI`.

4. game package

a. Game Class

This class has `GameGUI gameGUI`, `int STEP`, and `int[] previousScores` instances are used in this class to store respectively `currentGameGUI` which is given as an argument of constructor while creating its objects in `Home` class, player movement step for 15 ms, and the previous levels' final points between transitions. Other instances in this class are `Timer moveTimer` and `SecureRandom random`. The `moveTimer` is used to call the `movePlayer()` method one for every 15 ms, and the other one is used to only control randomly running states. These instances are initialized in the constructor and this object assign the public variable of `gameGUI` object which is `gameGUI.game` as well.

The `movePlayer()` method checks the two public boolean in `gameGUI` object which are related to direction and according to these booleans some other public variables related to player's coordinate and image in `gameGUI` object is changed to move player correctly. This method also checks the boundaries of the game to prevent the player leaving from the playing field.

The `addKeeper(KnowledgeKeepers keeper)` method adds the `KnowledgeKeepers` object which are used in this level into a public List in `gameGUI` object.

The `reset()` method reverts all changes during the game back to the initial state and stop the Timers.

The `recordScore(int score, int gameNum)` method saves the new final score into `src/users/scores.txt` with corresponding format.

The `gameDone()` method is called when the **Back** button is clicked in the game. This method appends required line into log file and calls `reset()` method. Then, it switches screen to home screen and calls `repaint()` and `revalidate()` methods for both `gameGUI` and `homeGUI` objects to prevent a bug that causes extra elements to be drawn on the screen.

The `gameDone(boolean status)` method which is overloaded version of previous one. The `status` parameter refers the winning situation, if `status` is `true`, the game is won; if it is not, the game is over. This method also appends required line into log file and calls `reset()` method and calls the `recordScore(previousScores[0]+previousScores[1]+Home.getPlayer().getScore(),Home.getPlayer().getGameNum())` method in addition to other version of this method. The others are same for both methods.

The `checkInteraction(KnowledgeKeepers keeper, ShotBoxes shotBox)` method is the most important method of the game. This method first checks the `shotBox` reach the floor with using coordinates. Then, it checks interaction between the `shotBox` and the player with coordinates. If interaction occurs, this method calls the `shotBox.interact(keeper)`, which is a **polymorphism** example with interface, and `Home.getPlayer().interact(shotBox)` methods to update required status in `shotBox` and `Home.getPlayer()` objects. Then, the question or info text of the interacted `shotBox` is written in the corresponding `JLabel` in the `gameGUI`. Also, all required updates fulfil to remove interacted `shotBox`. Finally it calls `checkStatus()` method.

The `checkStatus()` method just checks the required scores for the level transitions or whether the health reaches 0 which means game is over.

The `Game` class has **has-a relationship** with `GameGUI` class and **uses-a relationship** `Home`, `Player`, `KnowledgeKeepers`, `ShotBoxes`, `Informations`, `Questions`, and `MainGUI` classes.

b. Level1 Class

This class extends `Game` class and has a `List <KnowledgeKeepers>` `remainingSLs` instance to store the `SL` objects who are not in the game in level 1. In the `Level1` constructor, `new ArrayList<>(KnowledgeKeepers.getSLs())` is assigned to `remainingSLs`, then 4 random `SLs` are chosen in this `List` with using **inherited** `SecureRandom` object, and this chosen `SLs` are removed in the `remainingSLs` `List` after they are added in the game with **inherited** `addKeeper` method, and the necessary methods are called to start the `SL` movement. Finally required line is appended in the log file.

The `Level1` class has **is-a relationship** with `Game` class, and **uses-a relationship** with `KnowledgeKeepers`, `SL`, `Home`, and `Player` classes.

c. Level2 Class

This class extends `Game` class and has a `List <KnowledgeKeepers>` `remainingTAs` instance to store the TA objects who are not in the game in level 2. In the `Level2` constructor, firstly elements of `remainingSLs` in the `Level1` are added in the game with **inherited** `addKeeper` method, then new `ArrayList<> (KnowledgeKeepers.getTAs())` is assigned to `remainingTAs`, afterward 2 random TAs are chosen in this `List` with using **inherited** `SecureRandom` object, and this chosen TAs are removed in the `remainingTAs` `List` after they are added in the game with **inherited** `addKeeper` method, and the necessary methods are called to start the `KnowledgeKeepers` movement. Finally required line is appended in the log file.

The `Level2` class has **is-a relationship** with `Game` class, and **uses-a relationship** with `Level1`, `KnowledgeKeepers`, `SL`, `TA`, `Home`, and `Player` classes.

d. Level3 Class

This class extends `Game` class. In the `Level3` constructor, firstly elements of `remainingTAs` in the `Level2` are added in the game with **inherited** `addKeeper` method, then `Professor` objects in `KnowledgeKeepers.getProfessors()` are added in the game with **inherited** `addKeeper` method, and the necessary methods are called to start the `KnowledgeKeepers` movement. Finally required line is appended in the log file.

The `Level3` class has **is-a relationship** with `Game` class, and **uses-a relationship** with `Level2`, `KnowledgeKeepers`, `SL`, `Professor`, `Home`, and `Player` classes

e. GameGUI Class

This class extends `JPanel` class and implements `KeyListener` interface, and has `Game game`, `List<KnowledgeKeepers> keepers`, and some other instance variables.

In its constructor, GUI components are created such as `JButton`, `JLabel`, `JProgressBar` [6], and `JTextArea`, and current level's `Game` extended object is assigned to `game`. Also, required `Image` objects are created in the constructor. The **inherited** `addComponentListener` [7] method is also used to run some codes when this object opens, and in this method, all GUI components and game details are reset for new game, also, the `List<KnowledgeKeepers> keepers` `List`, whose elements are added with `addKeeper` method in the `Game` class, is being iterated in this method, and `startAttack()` method is called for all `KnowledgeKeepers` object in the `keepers` and it is a **polymorphism** example with abstract class.

The `paintComponen(Graphics g)` overridden method draws player, knowledge keepers, and shotboxes at the given coordinate while they are moving. Also, `game.checkInteraction` method is called in this method.

The `keyPressed(KeyEvent e)` overridden method checks whether a key on keyboard is pressed, and it makes `movingLeft` and `movingRight` booleans, which are responsible to player's movement direction, `true` according to pressed button.

The `keyReleased(KeyEvent e)` overridden method checks whether a key on keyboard is released, and it makes `movingLeft` and `movingRight` booleans `false` according to released button.

The `GameGUI` class has **is-a relationship** with `JPanel` class, **has-a relationship** with `Game` and `KnowledgeKeepers` classes, and **uses-a relationship** with `Informations`, `Questions`, `Home`, and `Player` classes.

5. player package

a. Player Class

This class is responsible to manage player's credentials. In the constructor, the files that have been created in registration and updated while the game is being played are read and the data is assigned to corresponding instance variables.

The `interact(ShotBoxes shotBox)` method increases score or decreases health according to actual type of `shotBox`, and the required line is written in the log file.

The `resetScoreHealth()` method make score 0 and health 100.

The `updateGameNum()` method updated the `gameNum.txt` file when a new play is done.

The `log(String msg, boolean newLine)` method helps to write new lines in the log file. A line is appended with the format `[msg] + " at " + [currentTime]` and a new line (`currentTime = new SimpleDateFormat("dd.MM.yyyy HH:mm:ss").format(new Date())`) [8]. Also, if the `newLine` is `true`, an additional new blank line added.

The `Player` class has **uses-a relationship** with `ShotBoxes`, `Informations`, and `Questions` classes.

6. knowledge_keepers package

a. KnowledgeKeepers (abstract) Class

This class is an abstract class and has static `List<KnowledgeKeepers>` instances named as `SLs`, `TAs`, `Professors` and `infos` and `quests` `List` whose elements

are `Informations` and `Questions` objects respectively. Additionally, some other instance variables exist.

When its constructor is called by one of its subclasses, first it assigns the parameters to the corresponding instance variables, then it appends itself into the corresponding `List` according to its actual type. For example, if it is a `SL` class object, it is appended into `SLs List`.

The `draw(Image keeperImage, Graphics g)` method is called in the `paintComponent` method in the `GameGUI` to draw the components related to `KnowledgeKeepers`. This method takes the `keeperImage`, which is the body image, since if it were created for each `KnowledgeKeepers` object, it would not be efficient, so I create it once in the `GameGUI` constructor.

The `attackQuestion()` method first check the `KnowledgeKeeper` already has a question, and if not choose a random question in the `quests List` with `SecureRandom` and while choosing random question, it also checks the chosen question is not currently in the game. Then `move` method for chosen `Questions` object is called.

The `attackInfo()` method first check the `KnowledgeKeeper` already has an info, and if not choose a random info in the `infos List` with `SecureRandom` and while choosing random info, it also checks the chosen info is not currently in the game. Then `move` method for chosen `Informations` object is called.

The `getCooridnate()` method which returns `int[]` array is not a standart getter method, since there is no coordinate array in the class. It returns only the x coordinates of the body image and face image of the `KnowledgeKeepers`, since their y coordinates are already fixed.

The `resetTimers()` method resets the timers and some variables related to `KnowledgeKeepers` when the `KnowledgeKeepers` objects are removed from the game.

Additionally, this class includes some abstract methods which will be used in `SL`, `TA`, and `Professor` classes.

The `KnowledgeKeepers` class has has-a relationship with `Questions` and `Informations` classes.

b. SL Class

This class extends `KnowledgeKeepers` class and when creating a `SL` instance, `easyInformations` and `easyQuestions` static Lists are gotten from `Informations` and `Questions` classes and assigned to inherited `infos` and `quests` instances after

applying `Collections.shuffle` method for randomness in this class, additionally, `Timers` related to SL's movement are created in the constructor.

The `startAttack()` method which is abstract in `KnowledgeKeepers` class starts a `Timer` whose time is randomly determined between **2000 and 4000 ms**. In this timer, the inherited `attackInfo` method is called with a rate of **60%** and the inherited `attackQuestion` method is called with a rate of **40%**.

The `move()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` whose time is 15 ms once it has been called, since the corresponding `Timer` starts in `startsAttack` method. This class provides the movement of the SLs according to `direction` boolean which is determined by `movementDirection()` method with an increase or decrease of **3 units** in the x coordinate every 15 ms. Also, boundaries are fixed in this method.

The `movementDirection()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` which has been started in constructor for the random time **500-1500 ms** once it has been called from elsewhere, since the `Timer` starts in itself. This method randomly chooses the value of `direction` boolean.

The `SL` class has **is-a relationship** with `KnowledgeKeepers`

c. TA Class

This class extends `KnowledgeKeepers` class and when creating a `TA` instance, `midInformations` and `midQuestions` static Lists are gotten from `Informations` and `Questions` classes and assigned to inherited `infos` and `quests` instances after applying `Collections.shuffle` method for randomness in this class, additionally, `Timers` related to TA's movement are created in the constructor.

The `startAttack()` method which is abstract in `KnowledgeKeepers` class starts a `Timer` whose time is randomly determined between **1250 and 2750 ms**. In this timer, the inherited `attackInfo` method is called with a rate of **50%** and the inherited `attackQuestion` method is called with a rate of **50%**.

The `move()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` whose time is 15 ms once it has been called, since the corresponding `Timer` starts in `startsAttack` method. This class provides the movement of the TAs according to `direction` boolean which is determined by `movementDirection()` method with an increase or decrease of **4 units** in the x coordinate every 15 ms. Also, boundaries are fixed in this method.

The `movementDirection()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` which has been started in constructor for the random time **500-1000 ms** once it has been called from elsewhere, since the `Timer` starts in itself. This method randomly chooses the value of `direction boolean` with a rate **60%**. With a rate **40%**, the `direction boolean` is determined towards to player's x coordinate.

The `TA` class has **is-a relationship** with `KnowledgeKeepers`.

d. Professor Class

This class extends `KnowledgeKeepers` class and when creating a `Professor` instance, `hardInformations` and `hardQuestions` static Lists are gotten from `Informations` and `Questions` classes and assigned to inherited `infos` and `quests` instances after applying `Collections.shuffle` method for randomness in this class, additionally, `Timers` related to Professor's movement are created in the constructor.

The `startAttack()` method which is abstract in `KnowledgeKeepers` class starts a `Timer` whose time is randomly determined between **1000 and 2000 ms**. In this timer, the inherited `attackInfo` method is called with a rate of **25%** and the inherited `attackQuestion` method is called with a rate of **75%**.

The `move()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` whose time is 15 ms once it has been called, since the corresponding `Timer` starts in `startAttack` method. This class provides the movement of the Professors according to `direction boolean` which is determined by `movementDirection()` method with an increase or decrease of **5 units** in the x coordinate every 15 ms. Also, boundaries are fixed in this method.

The `movementDirection()` method which is abstract in `KnowledgeKeepers` class is called with the `Timer` which has been started in constructor for the random time **250-750 ms** once it has been called from elsewhere, since the `Timer` starts in itself. This method randomly chooses the value of `direction boolean` with a rate **20%**. With a rate **80%**, the `direction boolean` is determined towards to player's x coordinate.

The `Professor` class has **is-a relationship** with `KnowledgeKeepers`.

7. shot_boxes package

a. ShotBoxes Interface

This interface has 3 `int[]` `questionsPoints`, `infoPoints`, and `speeds` arrays to provide constant values for points of the boxes and their speeds. Also, it has `interact`, `getCoordinate`, and `move` methods which will be defined in its subclasses.

b. informations subpackage

i. Informations (abstract) Class

This class which implements the `ShotBoxes` interface is an abstract class and has 3 `List<Informations>` named `easyInformations`, `midInformations`, and `hardInformations` and some other instances.

The static method `uploadInformations(String filePath)` which is called in main method read the file in given path which include the information's texts with the format in 2.A.1.c and creates new `Informations` extended object according to its difficulty and appends it into corresponding `List`. For example, If the line contains and easy information, a new `EasyInformations` object is created and appended in the `easyInformations` `List`.

The `draw(Image image, Graphics g)` method helps to draw image in the game. The `Image` object is given when the method is called, since the method is called in `GameGUI` class and it is created only once, but if it were created in `Informations` class, it would not be efficient.

The `interact(KnowledgeKeeper k)` method helps to stop its movement and remove itself in the game with changing required variables and stopping `Timer`.

The `Informations` class has **is-a relationship** with `ShotBoxes` and **uses-a relationship** with `KnowledgeKeepers`.

ii. EasyInformations Class

This class implements `ShotBoxes` interface and extends `Informations` class. In its constructor, given parameters are assigned to corresponding **inherited** instances only.

The overridden `move(KnowledgeKeepers k, int X)` method which is called when the object will be added to the game first check whether the object is in the game, if it is not, it is added to the game and changes the required variables, then starts a `Timer` with the time 25 ms, and this timer increase its Y coordinate 3 units if it hasn't reached the ground. If it has, the object is removed from the game.

The `EasyInformations` class has **is-a relationship** with `ShotBoxes` and `Informations` and **uses-a relationship** `KnowledgeKeepers`.

iii. MidInformations Class

In this class, only increasing step of Y is different from the `EasyInformations` class, which is 4. All the other codes are same.

iv. HardInformations Class

In this class, only increasing step of Y is different from the `EasyInformations` class, which is 5. All the other codes are same.

c. questions subpackage

i. Questions (abstract) Class

This class which implements the `ShotBoxes` interface is an abstract class and has 3 `List<Questions>` named `easyQuestions`, `midQuestions`, and `hardQuestions` and some other instances.

The static method `uploadQuestions(String filePath)` which is called in main method read the file in given path which include the information's texts with the format in 2.A.1.c and creates new `Question` extended object according to its difficulty and appends it into corresponding `List`. For example, If the line contains an easy question, a new `EasyQuestions` object is created and appended in the `easyQuestions` `List`.

The `draw(Image image, Graphics g)` method helps to draw image in the game. The `Image` object is given when the method is called, since the method is called in `GameGUI` class and it is created only once, but if it were created in `Questions` class, it would not be efficient.

The `interact(KnowledgeKeeper k)` method helps to stop its movement and remove itself in the game with changing required variables and stopping `Timer`.

The `Questions` class has **is-a relationship** with `ShotBoxes` and **uses-a relationship** with `KnowledgeKeepers`.

ii. EasyQuestions Class

This class implements `ShotBoxes` interface and extends `Questions` class. In its constructor, given parameters are assigned to corresponding **inherited** instances only.

The overridden `move(KnowledgeKeepers k, int X)` method which is called when the object will be added to the game first check whether the object is in the game, if it is not, it is added to the game and changes the required variables,

then starts a `Timer` with the time 25 ms, and this timer increase its Y coordinate 3 units if it hasn't reached the ground. If it has, the object is removed from the game.

The `EasyQuestions` class has **is-a relationship** with `ShotBoxes` and `Questions` and **uses-a relationship** `KnowledgeKeepers`.

iii. MidQuestions Class

In this class, only increasing step of Y is different from the `EasyQuestions` class, which is 4. All the other codes are same.

iv. HardQuestions Class

In this class, only increasing step of Y is different from the `EasyQuestions` class, which is 5. All the other codes are same.

8. exception package

a. InvalidAuthException Class

It is a **custom exception** that extends `RuntimeException` with the constructor that has a `String msg` argument. In the constructor it calls the `super(msg)`. It has **is-a relationship** with `RuntimeException`.

B. Files

1. assets folder

In this folder, there are the files to be used in-game and not to be modified.

a. project_image.png

The image on the cover of the project instruction to use as decoration on login and register pages.

b. questions.txt

The questions' texts with the format above.

```
Easy
What is Object-Oriented Programming (OOP)?
What is a class in Java?
...
Medium
What is polymorphism in OOP?
What is method overloading?
...
Hard
What are design patterns in OOP?
How does the SOLID principle relate to OOP?
...
```

c. informations.txt

The information's texts with the format above.

Easy

A class is a blueprint for creating objects in Java.

An object is an instance of a class.

...

Medium

Abstract classes can have both abstract and concrete methods.

Static methods belong to the class, not the object.

...

Hard

Java uses the JVM and garbage collection to manage object memory.

Interface-based programming promotes flexibility and testability.

...

d. game folder

It contains the images that are used in the game.

i. playerFront.png

The front view of the player

ii. playerLeft.png

The left view of the player

iii. playerRight.png

The right view of the player

iv. background.jpeg

The background image of the game

v. keeper.png

The fixed body image of the knowledge keepers

vi. question.png

An image with question mark to use as question shotbox

vii. info.png

An image with bulb to use as information shotbox

e. keepers folder

It has the knowledge keepers' images downloaded from the course website and named as `[name].[jpg|jpeg|png]`.

2. users folder

a. default.png

An image to use in the registration step when choosing no image

b. users.txt

A .txt file to store existing usernames and password with the format above

([username],[password])

```
bedirhankokduman,000000  
gürayk,gk20  
halilibo,halil123  
ahmetbugra,123456
```

c. scores.txt

Stores the final scores of the player with the format above

([score],[username],[gameNum])

```
200,bedirhankokduman,1  
300,gürayk,1  
310,halilibo,1  
120,gürayk,2  
230,bedirhankokduman,2  
200,halilibo,2  
320,ahmetbugra,1  
50,ahmetbugra,2  
310,ahmetbugra,3
```

d. [username] folder

This folder stores the corresponding player's credentials.

i. [username].log

In this .log file, player's logs are stored with the format as mentioned before in section 1.d and 2.c.iii.

ii. [username].[png | jpg | jpeg]

The chosen image when registration

iii. gameNum.txt

It stores only 1 number to count played game number.

References

- [1] <https://www.geeksforgeeks.org/java-swing-jfilechooser/>
- [2] <https://docs.oracle.com/javase/tutorial/uiswing/layout/card.html>
- [3] <https://docs.oracle.com/javase/tutorial/uiswing/components/dialog.html>
- [4] <https://www.geeksforgeeks.org/java-program-to-write-into-a-file/>
- [5] <https://stackoverflow.com/questions/72897755/how-to-include-a-jtable-and-some-text-below-the-table-in-joptionpane-showmessage>
- [6] <https://www.geeksforgeeks.org/java-swing-jprogressbar/>
- [7] <https://stackoverflow.com/questions/25338941/running-a-method-after-switching-cards-in-cardlayout>
- [8] <https://www.digitalocean.com/community/tutorials/java-simpledateformat-java-date-format>